

SKILL: taller-bv-dashboard-tech

Codigo exacto para extraer, aplicar y validar cambios en el Dashboard Taller BV

Rutas

- Dashboard (bash VM): /sessions/*/mnt/Taller BV/Dashboard_Pedidos.html
- XLSX descargado: /tmp/urgentes.xlsx
- Output subagente: /sessions/*/mnt/outputs/extraccion_resultado.json

Decodificar XLSX desde base64

```
import json, base64
with open('<ruta_del_txt>') as f:
    data = json.load(f)
raw = base64.b64decode(data['content'])
with open('/tmp/urgentes.xlsx', 'wb') as f:
    f.write(raw)
# Verificar: raw[:2] == b'PK' (es ZIP/XLSX)
```

Extraer produccion por GA (openpyxl)

```
import openpyxl
from datetime import datetime
from collections import defaultdict

wb = openpyxl.load_workbook('/tmp/urgentes.xlsx', data_only=True)

def extraer_ga(sheet_name):
    ws = wb[sheet_name]
    row1 = {c: ws.cell(1, c).value for c in range(1, ws.max_column+1)}
    row2 = {c: ws.cell(2, c).value for c in range(1, ws.max_column+1)}

    # Objetivos mensuales y semanales desde fila 1
    obj_mensual = {}; obj_sems = {}
    for c, v in row1.items():
        if isinstance(v, str):
            if 'Objetivo' in v and 'Jul' in v: obj_mensual['Jul'] = row1.get(c+1)
            if 'Objetivo' in v and 'Jun' in v: obj_mensual['Jun'] = row1.get(c+1)
            if 'Obj. S.' in v:
                try:
                    num = int(v.split('.')[0].strip())
                    val = row1.get(c+1)
                    if isinstance(val, (int, float)):
                        obj_sems[num] = round(val) # REDONDEAR AL ENTERO
                except: pass

    # Columnas julio (Estatores: ignorar 30/04)
    jul_cols = []
    for c, v in row2.items():
        if isinstance(v, datetime):
            if sheet_name == 'OF-EST' and v.month==4 and v.day==30: continue
            if v.month==7 and v.year==2026: jul_cols.append((c, v))

    # Produccion - EXCLUIR filas con codigo '-' o vacio (son totales)
    total = 0; codigos = []
    for row in range(3, ws.max_row+1):
        cod = ws.cell(row, 1).value
        if not cod or not isinstance(cod, str): continue
        cod = cod.strip()
        if cod in ('-', '', '-'): continue
        p = sum(ws.cell(row, c).value or 0 for c, _ in jul_cols)
        if isinstance(ws.cell(row, c).value, (int, float)):
            if p > 0:
                cat = ws.cell(row, 2).value
                total += p
                codigos.append({'codigo': cod, 'categoria': str(cat).strip() if cat else ''},
```

```

        'cantidad': int(p)})

# Semanas julio
sems = defaultdict(list)
for c, dt in sorted(jul_cols, key=lambda x: x[1]):
    sems[dt.isocalendar()[1]].append((c, dt))

semanas = []
for i, (wk, cols) in enumerate(sorted(sems.items())):
    prod_s = 0
    for row in range(3, ws.max_row+1):
        cod = ws.cell(row,1).value
        if not cod or not isinstance(cod,str) or cod.strip() in('-',',','-'): continue
        prod_s += sum(ws.cell(row,c).value or 0 for c,_ in cols
                      if isinstance(ws.cell(row,c).value,(int,float)))
    semanas.append({'num':i+1, 'dias':len(cols), 'obj':obj_sems.get(i+1,0), 'prod':int(prod_s)})

return {'obj_mensual_jul': round(obj_mensual.get('Jul') or 0),
        'dias_habiles_jul': len(jul_cols), 'prod_jul': int(total),
        'semanas_jul': semanas, 'cod_jul': codigos}

ind = extraer_ga('OF-IND')
rot = extraer_ga('OF-ROT')
est = extraer_ga('OF-EST')

```

Extraer pedidos (hoja Urgentes)

```

ws = wb['Urgentes']
# Col 1=N, 2=Ingreso, 3=GA, 4=Cod, 5=Cond, 6=Cliente
# Col 7=Dias obj, 8=Entrega sol, 12=Reclamo, 13=Obs
# Col 14=Entrega real, 16=Dias prod. (P), 17=Demora (Q)

def fmt_date(v):
    if isinstance(v, datetime): return v.strftime('%Y-%m-%d')
    if isinstance(v, str) and v not in ('-',','): return v
    return None

pedidos_sheet = {}
for row in range(3, ws.max_row+1):
    n = ws.cell(row,1).value; ga = ws.cell(row,3).value
    if not isinstance(n,(int,float)) or not isinstance(ga,str): continue
    n = int(n)
    dias_prod_raw = ws.cell(row,16).value
    # dias_prod negativo = artifact de formula → null
    dias_prod = int(dias_prod_raw) if isinstance(dias_prod_raw,(int,float)) and dias_prod_raw >= 0 else None
    pedidos_sheet[n] = {
        'n': n, 'ga': str(ga).strip(),
        'codigo': str(ws.cell(row,4).value).strip() if ws.cell(row,4).value else None,
        'condicion': ws.cell(row,5).value, 'cliente': ws.cell(row,6).value,
        'dias_objetivo': int(ws.cell(row,7).value) if isinstance(ws.cell(row,7).value,(int,float)) else None,
        'entrega_sol': fmt_date(ws.cell(row,8).value),
        'reclamo': ws.cell(row,12).value if ws.cell(row,12).value not in (None,'-') else None,
        'observacion': ws.cell(row,13).value if ws.cell(row,13).value not in (None,'-') else None,
        'fecha_entrega_real': fmt_date(ws.cell(row,14).value),
        'dias_prod': dias_prod,
        'demora': round(float(ws.cell(row,17).value),4) if isinstance(ws.cell(row,17).value,(int,float)) else None,
    }
}

```

Actualizar badge

```

import re
from datetime import datetime, timezone, timedelta
AR = timezone(timedelta(hours=-3))
ahora = datetime.now(AR)
dias = ['Lun', 'Mar', 'Mié', 'Jue', 'Vie', 'Sáb', 'Dom']
fecha_str = f"{dias[ahora.weekday()]}" {ahora.strftime('%d/%m/%Y %H:%M')}
html = re.sub(r"const _ULTIMO_SYNC='[^']*'",
              f"const _ULTIMO_SYNC='{fecha_str}'", html)
# CRITICO: usar re.sub, NO html.find(";")

```

Validacion post-aplicacion (Node.js)

```
node -e "
const fs=require('fs');
const html=fs.readFileSync('/sessions/*/mnt/Taller BV/Dashboard_Pedidos.html','utf8');
const ofStart=html.indexOf('const OF_DATA=');
const pedEnd=html.indexOf('];',html.indexOf('const PEDIDOS=')+2);
try {
  const {OF_DATA,PEDIDOS}=new Function(
    html.slice(ofStart,html.indexOf('];',ofStart)+2)+'\n'+
    html.slice(html.indexOf('const PEDIDOS='),pedEnd)+'\nreturn {OF_DATA,PEDIDOS};')();
  console.log('OF_DATA:',OF_DATA.length,'| PEDIDOS:',PEDIDOS.length);
  ['Inducidos','Rotores','Estatores'].forEach(ga=>{
    const d=OF_DATA.find(x=>x.ga===ga&&x.mes_key==='2026-07');
    if(d) console.log(ga,'Jul: obj='+d.obj_mensual+' prod='+d.prod_total);
  });
  console.log('JS OK');
} catch(e){ console.log('ERROR:',e.message); }
"
```

No sobrecribir con vacios

```
def es_cambio(campo, v_dash, v_sheet):
  if v_sheet is None: return False # no sobrecribir con vacio
  if v_dash == v_sheet: return False
  if campo == 'demora' and v_dash is not None:
    if round(float(v_dash),2) == round(float(v_sheet),2): return False
  if campo == 'cliente':
    if str(v_dash or '').lower() == str(v_sheet or '').lower(): return False
  return True
```